

Описание на курсов проект за втори срок по предмета „Операционни системи“

Izgotvil: Mladen Mihalkov – 11g klas

Съдържание:

1. Основна задача на проекта
2. Описание на задачите
 - 2.1. Нех dump
 - 2.2. Обръщане на файл на място
 - 2.3. Паралелно изпълнение на команди
 - 2.4. Мини-шел с тръба
 - 2.5. Изящно спиране
 - 2.6. Времеви сървър
 - 2.7. Ехо-сървър с конкурентни клиенти

1. Основна задача на проекта

Целта на проекта е разработчика да приложи на практика всички умения, които е придобил по време на втория срок по предмета „Операционни системи“ за работа със системни извиквания от ниско ниво и работа с програмния език C.

Той е разбит на множество подзадачи, всяка от които е с приложен характер и изисква комбинирането на различни функционалности.

Общо задачите обхващат различни важни части от системното програмиране в средата Linux като: мрежово програмиране, работа с файлове, манипулиране на потоци от данни(stdin, stdout, stderr), работа с тръби, работа с процеси, междупроцесна комуникация, системни прекъсвания.

В този документ е описана реализацията на всяка една от тези задачи подробно.

2. Описание на задачите:

2.1 Hex dump

Описание: Програма, която взима текстови fail и представя съдържанието му в шеснадесетична система в конзолата.

```
int main(int argc, char *argv[]) {
    // Проверка за подаден аргумент (път към файл)
    if (argc != 2) {
        // Извеждане на съобщение за грешка към stderr(използвам write вместо fprintf).
        write(2, "Usage: ./hexdump <file>\n", 24);
        return 1;
    }

    // Отваряне на файла само за четене
    int fd = open(argv[1], O_RDONLY);
    if (fd < 0) { // проверка дали файла е бил отворен успешно
        perror("hexdump");
        return 1;
    }
}
```

Фиг. 2.1.1

V началото на main функцията проверявам дали е бил подаден само един аргумент - име на fail. Ако не е, извеждам грешка към stderr, описвайки правилния начин за използване на командата. Тогаваш програмата приключва с код 1.

Ако е подаден аргумент, програмата се опитва да отвори fail само в режим на четене с пътя, оказан от аргумента.

След това се прави проверка дали това е било успешно.

Ако не е, с perror се извежда грешка и програмата приключва със статус код 1.

Инициализират се следните променливи:

```

unsigned char buffer[16]; // буфер за директен запис на съдържание от файл
char out_buf[80]; // Буфер за един ред (достатъчно голям за hex и ASCII частта)
const char hex_digits[] = "0123456789ABCDEF"; // масив с цялата шестнадесетична азбука
ssize_t bytes_read; // променлива, съхраняваща брой прочетени байтове

```

Фиг. 2.1.2

Предназначението им е обяснено в коментарите.

```

// Четене на парчета от по най-много 16 байта докато има съдържание във файла.
while ((bytes_read = read(fd, buffer, sizeof(buffer))) > 0) {
    int out_idx = 0; // до къде е запълнен изходния буфер

    // 1. Шеснадесетичен запис на съдържанието на един ред от файла в изходния буфер out_buf
    for (int i = 0; i < 16; i++) {
        if (i < bytes_read) {
            // Преобразуване на байта в 2 hex знака
            out_buf[out_idx++] = hex_digits[buffer[i] >> 4]; // запис на първи шестнадесетичен символ
            out_buf[out_idx++] = hex_digits[buffer[i] & 0x0F]; // запис на втори шестнадесетичен символ
        } else {
            // Ако байтът липсва или е достигнат край на файла, запълваме с разстояния
            out_buf[out_idx++] = ' ';
            out_buf[out_idx++] = ' ';
        }
    }

    // Добавяне на интервал между всеки байт (без след последния)
    if (i < 15) {
        out_buf[out_idx++] = ' ';
    }

    // Добавяне на допълнителен интервал между 8-ми и 9-ти байт
    if (i == 7) {
        out_buf[out_idx++] = ' ';
    }
}

```

Фиг. 2.1.1

След това започва да се попълва изходния буфер. От файла се четат парчета от най-много 16 байта. Броя прочетени байтове се записва в `bytes_read`.

Докъто не са превърнати всички прочетени байтове в шестнадесетични символи, те се превръщат като се използва hex_digits масива.

След всеки прочетен байт се поставя по един празен интервал. Ако са били прочетени 8 символа, се слага и един допълнителен.

Индекса на буфера се увеличава а с едно при всяко записва не в него.

Ако броя байтове е по-малък от 16, всеки липсващ байт се запълва с две празни места.

```
// 3. Запис на същия ред в ASCII формат в изходния буфер
for (int i = 0; i < bytes_read; i++) {
    // Проверка дали знакът може да бъде визуализиран (между 32 и 127 вкл.)
    if (buffer[i] >= 32 && buffer[i] <= 127) {
        out_buf[out_idx++] = buffer[i];
    } else {
        out_buf[out_idx++] = '.'; // Ако не се записва точка
    }
}

// 4. Запис на символ за нов ред в края на всеки ред на буфера
out_buf[out_idx++] = '\n';

// извежда целия изходен буфер в стандартния изход
write(1, out_buf, out_idx);
}
```

След като всички 16 символа са записани в изходния буфер, там се записват същите 16 байта но в ASCII формат.

За всеки един байт се проверява дали той може да бъде визуализиран на екрана. Ако не, се поставя точка в изходния буфер.

На края на реда в този буфер се поставя символ да нов ред. Целия изходен буфер се изпечатва на стандартния изход. След това този процес се повтаря докато има останали непрочетени байтове във файла.

```
// Проверка дали е имало грешки при четенето.  
if (bytes_read < 0) {  
    perror("hexdump");  
    close(fd);  
    return 1;  
}  
  
close(fd); // Затваряне на файлов дескриптор  
return 0;  
}
```

Най накрая се проверява дали не са били възникнали някакви проблеми при последното четене от файла. Ако да, се извежда грешка и програмата приключва с изходен код 0. В противен случай файловия дескриптор се освобождава и програмата прислужва успешно със статус код 0.

2.2 Обръщане на файл на място

Описание: Програма, която приема път към файл като аргумент и обръща огледално съдържанието му бит по бит без да го зарежда целия в паметта.

```

int main(int argc, char *argv[]) {
    // Проверка дали е подаден точно един аргумент към програмата.
    if (argc != 2) {
        // Извеждам пояснение как се използва тя през stderr.
        write(2, "Usage: ./reverse <file>\n", 24);
        return 1;
    }

    // Отваряне на файла за четене и писане
    int fd = open(argv[1], O_RDWR);
    if (fd < 0) { // проверка дали е бил отворен успешно
        perror("reverse");
        return 1;
    }
}

```

Фиг 2.2.1

Първо проверявам дали на програмата е подаден само един аргумент. Ако не, извежда пояснение за как трябва да бъде използвана програмата на stderr и програмата приключва със статус код 1.

```

// Намиране на размера на файла
off_t left = 0; // променлива, показваща до къде съм стигнал от лявата граница на файла
off_t right = lseek(fd, 0, SEEK_END); // променлива, показваща до къде съм стигнал от дясната граница на файла
if (right < 0) { // при грешка с lseek
    perror("reverse");
    close(fd);
    return 1;
}

```

фиг. 2.2.2

Създавам два индекса, които ще определят дясна и лява граница в файла. Левият индекс е инициализиран на 0, а десния – с последния байт във файла.

Ако е възникнала грешка, тя се извежда и програмата завършва с 1.

```

// Междинен буфер за обръщане на съдържанието на файла
unsigned char buffer[512];

// Придвижване от двата края на файла към средата
while (left < right) {
    off_t remaining = right - left; // променлива, съхраняваща броя останали необърнати байтове във файла

    // Максималния брой байтове от едната страна на файла, които да бъдат записани в буфера.
    size_t chunk = (remaining / 2 > 256) ? 256 : (size_t)(remaining / 2);

    // Ако остава само 1 байт в точната среда на файла, няма нужда от размяна
    if (chunk == 0) {
        break; // програмата напуска цикъла
    }
}

```

фиг. 2.2.3

След това създавам междинен буфер, в който ще извършвам манипулациите над байтовете.

Тогава програмата влиза в цикъл доокъто лявата и дясната граница не се „срещнат“ и „задминат“.

При всяка итерация се изчислява броя на останалите необърнати байтове във файла.

След това се намира число наречено **chunk**. То показва какъв брой байтове да се прочетат от лявата и дясната граница и да се заредят в буфера.

Буфера има фиксирана големина от 512 байта. При всяка итерация в него се зарежда както част от лявата така и от дясната страна. Следователно максимално могат да се заредят по 255 байта за всяка от страните.

Ако броя на оставащите битове е толкова голям, тогава това е максималния брой, който да бъде записан в буфера.

Ако пък оставащите брой битове са по-малко, тогава размера ще е $\frac{remaining}{2}$. Ако остава само един байт, то тогава chunk ще бъде 0. Тъй като в този случай няма какво да бъде обръщано, програмата излиза от цикъла.

```

// 1. Прочитане на брой битове, посочени от chunk, от лявата страна на файла и запис в buffer.
if (lseek(fd, left, SEEK_SET) < 0 || // Преместване на курсора в началото на файла
    read(fd, buffer, chunk) != (ssize_t)chunk) {
    perror("reverse"); // При възникнала грешка
    close(fd);
    return 1;
}

// 2. Прочитане на брой битове, посочени от chunk, от дясната страна на файла и запис в buffer.
if (lseek(fd, right - chunk, SEEK_SET) < 0 ||
    read(fd, buffer + 256, chunk) != (ssize_t)chunk) {
    perror("reverse");
    close(fd);
    return 1;
}

// 3. Размяна на байтовете между лявата и дясната част на буфера
for (size_t i = 0; i < chunk; i++) {
    unsigned char temp = buffer[i]; // Байт от лявото парче
    // Разменяме го с огледалния му байт от дясното парче
    buffer[i] = buffer[256 + chunk - 1 - i];
    buffer[256 + chunk - 1 - i] = temp;
}

// 4. Запис на лявата част на буфера в лявата част на файла
if (lseek(fd, left, SEEK_SET) < 0 ||
    write(fd, buffer, chunk) != (ssize_t)chunk) {
    perror("reverse");
    close(fd);
    return 1;
}

// 5. Запис на дясната част на буфера в дясната част на файла
if (lseek(fd, right - chunk, SEEK_SET) < 0 ||
    write(fd, buffer + 256, chunk) != (ssize_t)chunk) {
    perror("reverse");
    close(fd);
    return 1;
}

```

Фиг. 2.2.4

След това в лявата част на буфера се записват толкова брой байтове от лявата част на файла ,колкото е оказано от променливата chunk. После се повтаря същото, но

за дясната част на буфера и файла. След като съдържанието в буфера е попълнено байтовете в лявата и дясната му страна си разменят местата. После лявата част на буфера се записва към лявата страна на файла на дясната част на буфера се записва към дясната част на файла. Тук отново при възникване на някаква грешка тя се извежда и програмата завършва със статус код единица.

2.2.4

След
тези

```
// Преместване на границите с броя обърнати байтове
left += chunk;
right -= chunk;
}

close(fd); // затваряне на файловия дескриптор
return 0;
}
```

Фиг

ВСИЧКИ

операции, лявата и дясна граница се придвижват с броя байтове, посочен от променливата , chunk и така цикълът отново се повтаря докато целият файл не бъде обърнат. Най-накрая файловият дискрептор се затваря и програмата завършва успешно със статус код 0.

2.3 Паралелно изпълнение на команди

Описание: Програма, която приема като аргументи множество на брой Линукс команди и ги изпълнява едновременно в различни отделни процеси.

Тя ги изчаква да приключат и накрая извежда статус кодът с който са приключили изпълнението си. Ако е имало грешка по време на тяхното изпълнение, тя ще бъде изведена в конзолата.

```
int main(int argc, char *argv[]) {
    // Ако не са подадени команди като аргументи, програмата приключва успешно
    if (argc < 2) {
        return 0;
    }

    // Масив, в който запазваме PID-овете на създадените процеси,
    // за да можем след това да свържем всеки PID с оригиналния му команден низ
    pid_t pids[argc];
```

Фиг 2.3.1

Проверка дали са подадени аргументи към програмата.

```

// За да можем след това да свържем всеки PID с оригиналния процес
pid_t pids[argc];

// 1. Стартиране на всички процеси паралелно
for (int i = 1; i < argc; i++) {
    pids[i] = fork();
}

```

Създаване на масив, в който ще бъдат записвани pid на отделните дъщерни процеси.

След това в цикъл един след един се създават всички отделни процеси.

```

} else if (pids[i] == 0) {
    // ----- ДЪЩЕРЕН ПРОЦЕС -----

    // Заглушаваме стандартния изход на командата (STDOUT),
    // за да не се смесва с финалния статус, очакван от тестовите
    int devnull = open("/dev/null", O_WRONLY);
    if (devnull >= 0) {
        dup2(devnull, STDOUT_FILENO);
        close(devnull);
    }

    // Правим копие на низа, тъй като strtok го модифицира на място
    char *cmd_copy = strdup(argv[i]);
    if (!cmd_copy) {
        perror("prun");
        exit(1);
    }

    char *args[256];
    int arg_count = 0;
}

```

Фиг 2.3.3

Ако този код се изпълнява от дъщерен процес, неговият stdout ще бъде пренасочен към /dev/null така че да не се смесва с изхода от моята програма.

Копираме текущата команда от аргументния масив.

```
// Нарязваме низа на отделни аргументи по интервали и табулации
char *token = strtok(cmd_copy, " \\t\\n");
while (token != NULL && arg_count < 255) {
    args[arg_count++] = token;
    token = strtok(NULL, " \\t\\n");
}
args[arg_count] = NULL; // Задължителен завършващ NULL елемент за execvp

// Ако командата не е празен низ, я изпълняваме
if (arg_count > 0) {
    execvp(args[0], args);
    // Ако execvp върне управлението, означава, че е възникнала грешка (напр. командата липсва)
    perror("prun");
}

// Прекратяваме дъщерния процес с код 1 при неуспешен execvp
free(cmd_copy);
exit(1);
```

Тоуканизираме аргументите така че да бъдат в правилен формат, готови за изпращане към `execvp`. След това извикваме `execvp`.

```

// 2. Изчакване на приключването на всички дъщерни процеси
int status;
pid_t wpid;

// waitpid(-1, ...) прихваща процесите в реалния ред на тяхното завършване
while ((wpid = waitpid(-1, &status, 0)) > 0) {

    // Намираме индекса на завършилия процес в нашия масив `pids`
    int idx = -1;
    for (int i = 1; i < argc; i++) {
        if (pids[i] == wpid) {
            idx = i;
            break;
        }
    }

    // Ако процесът е намерен, извличаме кода му и го принтираме
    if (idx != -1) {
        int exit_code = -1;

        if (WIFEXITED(status)) {
            // Процесът е завършил нормално
            exit_code = WEXITSTATUS(status);
        } else if (WIFSIGNALED(status)) {
            // Процесът е бил прекратен от системен сигнал
            exit_code = 128 + WTERMSIG(status);
        }

        // Отпечатване на резултата на стандартния изход
        printf("[%d] \"%s\" exited with status %d\n", wpid, argv[idx], exit_code);
    }
}

return 0;
}

```

След това изчакваме всички процеси да завършат и за всеки един, извеждаме неговият изходен код.

2.4 Мини-шел с тръба

```
// Помощна функция за парсване на командния низ (като в prun)
void parse_cmd(char *cmd_str, char **args, int max_args) {
    int arg_count = 0;
    char *token = strtok(cmd_str, " \t\n");
    while (token != NULL && arg_count < max_args - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t\n");
    }
    args[arg_count] = NULL;
}
```

Функция, която приема cmd str, тоуканизира го и записва тоукъните в args.

```
// Проверка за точно 2 аргумента (името на програмата + 2 команди)
if (argc != 3) {
    write(2, "Usage: ./pipe2 <cmd1> <cmd2>\n", 29);
    return 1;
}

int pipefd[2];
// Създаване на анонимна тръба
if (pipe(pipefd) < 0) {
    perror("pipe2");
    return 1;
}
```

Проверява дали са подадени точно два аргумента.

```

// --- СТАРТИРАНЕ НА ПЪРВАТА КОМАНДА (cmd1) ---
pid_t pid1 = fork();
if (pid1 < 0) {
    perror("pipe2");
    return 1;
} else if (pid1 == 0) {
    // Дъщерен процес 1: Пише в тръбата
    // Насочваме STDOUT към входа за писане на тръбата (pipefd[1])
    if (dup2(pipefd[1], STDOUT_FILENO) < 0) {
        perror("pipe2");
        exit(1);
    }
    // Затваряме вече ненужните за детето копия на дескрипторите
    close(pipefd[0]);
    close(pipefd[1]);

    // Парсваме и изпълняваме
    char *cmd1_copy = strdup(argv[1]);
    char *args1[256];
    parse_cmd(cmd1_copy, args1, 256);

    if (args1[0] != NULL) {
        execvp(args1[0], args1);
        perror("pipe2"); // Извиква се само при грешка в execvp
    }
    free(cmd1_copy);
    exit(1);
}

```

Стартира се първата команда и изхода и се пренасочва към изхода на тръбата.

```

// --- СТАРТИРАНЕ НА ВТОРАТА КОМАНДА (cmd2) ---
pid_t pid2 = fork();
if (pid2 < 0) {
    perror("pipe2");
    return 1;
} else if (pid2 == 0) {
    // Дъщерен процес 2: Чете от тръбата
    // Насочваме STDIN да взима данни от изхода на тръбата (pipefd[0])
    if (dup2(pipefd[0], STDIN_FILENO) < 0) {
        perror("pipe2");
        exit(1);
    }
    // Затваряме ненужните копия
    close(pipefd[0]);
    close(pipefd[1]);

    // Парсваме и изпълняваме
    char *cmd2_copy = strdup(argv[2]);
    char *args2[256];
    parse_cmd(cmd2_copy, args2, 256);

    if (args2[0] != NULL) {
        execvp(args2[0], args2);
        perror("pipe2");
    }
    free(cmd2_copy);
    exit(1);
}

```

Създаваме втория процес и свързваме неговия stdin към края на тръбата – ефективно свързвам изхода на едната към кода на другата.

```

// --- БАЩИН ПРОЦЕС ---
// КРИТИЧНО: Бащата затваря своите краища на тръбата, за да може cmd2 да получи EOF,
// когато cmd1 приключи да пише!
close(pipefd[0]);
close(pipefd[1]);

int exit_code = 0;

// Изчакаме и двата процеса без значение кой ще завърши първи
pid_t wpid;
int status;
while ((wpid = waitpid(-1, &status, 0)) > 0) {
    if (wpid == pid2) {
        // Директно извличаме статуса на дясната команда (cmd2) от 'status'
        if (WIFEXITED(status)) {
            exit_code = WEXITSTATUS(status);
        } else if (WIFSIGNALED(status)) {
            exit_code = 128 + WTERMSIG(status);
        }
    }
}

return exit_code;
}

```

Изчакаме и двата процеса. След като втория приключи взимаме изхода му и го записваме в конзолата.

2.5 Изящно спиране

```
volatile sig_atomic_t should_exit = 0;
volatile sig_atomic_t status_requested = 0;
```

Тези променливи са атомни – операциите се извършват като транзакция. Те са също от тип `volatile` – следователно не подлежат на оптимизации на компилатора.

```
void handle_signal(int sig) { // извиква се при сигнал
    if (sig == SIGUSR1) {
        status_requested = 1;
    } else {
        should_exit = 1;
    }
}
```

Тази функция се извиква при всяко подаване на сигнал към програмата. В зависимост от Получения сигнал, се задава съответната променлива.

```
int counter = 1;

while (!should_exit) {
    printf("tick %d\n", counter);
    fflush(stdout);

    sleep(1);

    if (status_requested) {
        status_requested = 0;
        printf("status: tick %d\n", counter);
        fflush(stdout);
    }

    if (!should_exit) {
        counter++;
    }
}

printf("shutdown after %d ticks\n", counter);
return 0;
}
```

Тук програмата влиза в цикъл и в зависимост кой флаг е бил зададен се изпълнява съответната реакция.

2.6 Времеви сървър

Timeserver.c:

```
if (argc != 2) {
    write(2, "Usage: ./timeserver <port>\n", 27);
    return 1;
}

int port = atoi(argv[1]);
if (port <= 0 || port > 65535) {
    write(2, "timeserver: Invalid port\n", 25);
    return 1;
}
```

Проверка дали е подаден порт като аргумент и ако да дали е валиден.

```

// 1. Създаване на TCP сокет
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd < 0) {
    perror("timeserver");
    return 1;
}

// 2. Активиране на SO_REUSEADDR опцията
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
    perror("timeserver");
    close(server_fd);
    return 1;
}

// 3. Структура за адрес (0.0.0.0:<port>)
struct sockaddr_in address;
memset(&address, 0, sizeof(address));
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY; // Свързва се към 0.0.0.0
address.sin_port = htons(port);

// 4. Bind (свързване) на сокета към адреса
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("timeserver");
    close(server_fd);
    return 1;
}

```

Тук сокета на сървъра се свързва с IP адрес, адресно семейство и др.

```

// 5. Listen за входящи връзки (опашка от 1)
if (listen(server_fd, 1) < 0) {
    perror("timeserver");
    close(server_fd);
    return 1;
}

// 6. Приемане на една входяща връзка
int client_fd = accept(server_fd, NULL, NULL);
if (client_fd < 0) {
    perror("timeserver");
    close(server_fd);
    return 1;
}

```

Изчакване само за една връзка със сървъра.

```

// 7. Вземане и форматиране на текущото местно време
time_t raw_time = time(NULL);
struct tm *time_info = localtime(&raw_time);

char time_buffer[64];
// Формат: YYYY-MM-DD HH:MM:SS\n
size_t len = strftime(time_buffer, sizeof(time_buffer), "%Y-%m-%d %H:%M:%S\n", time_info);

// 8. Изпращане на времето към клиента
if (len > 0) {
    write(client_fd, time_buffer, len);
}

// 9. Затваряне на връзките и изход
close(client_fd);
close(server_fd);
return 0;

```

Взимане на текущото време, форматирането му в правилния формат и изпращането му към клиента.

След това програмата на сървъра прекъсва.

Timeclient.c:

```
if (argc != 3) {
    write(2, "Usage: ./timeclient <hostname> <port>\n", 38);
    return 1;
}

char *hostname = argv[1];
int port = atoi(argv[2]);
if (port <= 0 || port > 65535) {
    write(2, "timeclient: Invalid port\n", 25);
    return 1;
}
```

Проверка дали hostname и port са подадени като аргументи и ако да дали са правилни.

```

// 1. Създаване на TCP сокет
int sock_fd = socket(AF_INET, SOCK_STREAM, 0);
if (sock_fd < 0) {
    perror("timeclient");
    return 1;
}

// 2. Намиране на хоста (поддържа IP-та като 127.0.0.1 или имена като localhost)
struct hostent *server = gethostbyname(hostname);
if (server == NULL) {
    write(2, "timeclient: Host not found\n", 27);
    close(sock_fd);
    return 1;
}

struct sockaddr_in server_address;
memset(&server_address, 0, sizeof(server_address));
server_address.sin_family = AF_INET;
memcpy(&server_address.sin_addr.s_addr, server->h_addr_list[0], server->h_length);
server_address.sin_port = htons(port);

```

Създаване на socket.

```

// 3. Свързване (connect) към сървъра
if (connect(sock_fd, (struct sockaddr *)&server_address, sizeof(server_address)) < 0) {
    perror("timeclient");
    close(sock_fd);
    return 1;
}

// 4. Четене на отговора до затваряне на връзката (EOF)
char buffer[256];
ssize_t bytes_read;
while ((bytes_read = read(sock_fd, buffer, sizeof(buffer))) > 0) {
    write(1, buffer, bytes_read); // Извеждане директно на стандартния изход
}

if (bytes_read < 0) {
    perror("timeclient");
    close(sock_fd);
    return 1;
}

// 5. Затваряне и изход с код 0
close(sock_fd);
return 0;
}

```

Свързване към клиента и прочитане на отговора на сървъра.

2.6 Ехо-сървър с конкурентни клиенти

Echoclient.c:

```
if (argc != 4) {
    write(2, "Usage: ./echoclient <hostname> <port> <message>\n", 48);
    return 1;
}

char *hostname = argv[1];
int port = atoi(argv[2]);
char *message = argv[3];

if (port <= 0 || port > 65535) {
    write(2, "echoclient: Invalid port\n", 25);
    return 1;
}
```

Проверка дали са подадени правилния брой аргументи и дали те са валидни.

```

// 1. Създаване на TCP сокет
int sock_fd = socket(AF_INET, SOCK_STREAM, 0);
if (sock_fd < 0) {
    perror("echoclient");
    return 1;
}

// 2. Резолване на хоста
struct hostent *server = gethostbyname(hostname);
if (server == NULL) {
    write(2, "echoclient: Host not found\n", 27);
    close(sock_fd);
    return 1;
}

struct sockaddr_in server_address;
memset(&server_address, 0, sizeof(server_address));
server_address.sin_family = AF_INET;
memcpy(&server_address.sin_addr.s_addr, server->h_addr_list[0], server->h_length);
server_address.sin_port = htons(port);

```

Създаване на сокет и свързването му с IP адрес.

```

// 3. Свързване към ехо-сервър
if (connect(sock_fd, (struct sockaddr *)&server_address, sizeof(server_address)) < 0) {
    perror("echoclient");
    close(sock_fd);
    return 1;
}

// 4. Изпращане на съобщението
size_t msg_len = strlen(message);
size_t total_sent = 0;
while (total_sent < msg_len) {
    ssize_t sent = write(sock_fd, message + total_sent, msg_len - total_sent);
    if (sent <= 0) {
        perror("echoclient: write");
        close(sock_fd);
        return 1;
    }
    total_sent += sent;
}

// Затваряме изхода на сокета (shutdown writing), за да покажем на сервъра,
// че сме приключили с пращането, но оставяме входа отворен за четене.
shutdown(sock_fd, SHUT_WR);

```

Свързване със сервъра и изпращане на съобщение.

```

// 5. Четене на ехо отговора и отпечатване на стандартния изход
char buffer[1024];
ssize_t bytes_read;
while ((bytes_read = read(sock_fd, buffer, sizeof(buffer))) > 0) {
    write(1, buffer, bytes_read);
}

// Добавяме нов ред в края за прегледност на конзолата
write(1, "\n", 1);

if (bytes_read < 0) {
    perror("echoclient: read");
    close(sock_fd);
    return 1;
}

// 6. Затваряне и изход
close(sock_fd);
return 0;

```

Прочитане на ехо отговора.

Echoserver.c

```

// Асинхронно безопасен обработчик за SIGCHLD сигнала
void handle_sigchld(int sig) {
    (void)sig;
    // Прибираме всички умрели дъщерни процеси (зомбита) наведнъж
    // WNOHANG гарантира, че waitpid няма да блокира, ако няма повече завършили деца
    while (waitpid(-1, NULL, WNOHANG) > 0);
}

```

Тази функция се извиква получен сигнал SIGCHLD. Тогава сървър изчаква всички клиенти да приключат изпълнението си и след това програмата приключва.

```
// 1. Настройка на маската и обработчика за SIGCHLD
struct sigaction sa;
sa.sa_handler = handle_sigchld;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART; // Автоматично рестартира assert(), ако бъде прекъснат от сигнал
if (sigaction(SIGCHLD, &sa, NULL) < 0) {
    perror("echoserver: sigaction");
    return 1;
}
```

Настройка на обработчика.

```

// 2. Създаване на TCP сокет
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd < 0) {
    perror("echoserver");
    return 1;
}

// 3. Опция SO_REUSEADDR
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
    perror("echoserver");
    close(server_fd);
    return 1;
}

struct sockaddr_in address;
memset(&address, 0, sizeof(address));
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY; // 0.0.0.0
address.sin_port = htons(port);

// 4. Bind и Listen
if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("echoserver");
    close(server_fd);
    return 1;
}

```

Създаване на socket.

```

if (listen(server_fd, 10) < 0) {
    perror("echoserver");
    close(server_fd);
    return 1;
}

// Безкраен цикъл за приемане на конкурентни клиенти
while (1) {
    int client_fd = accept(server_fd, NULL, NULL);
    if (client_fd < 0) {
        // Ако accept е прекъснат от сигнал, просто опитваме пак
        if (errno == EINTR) {
            continue;
        }
        perror("echoserver: accept");
        continue;
    }

    // Създаване на нов процес за новия клиент
    pid_t pid = fork();
    if (pid < 0) {
        perror("echoserver: fork");
        close(client_fd);
    }
}

```

Влизане в безкраен цикъл за приемане на заявки.

```

} else if (pid == 0) {
    // ----- ДЪЩЕРЕН ПРОЦЕС (ОБРАБОТЧИК) -----
    close(server_fd); // Детето няма нужда от слушащия сокет

    char buffer[1024];
    ssize_t bytes_read;

    // Четем на блокове от 1024 байта и пращаме обратно дословно
    while ((bytes_read = read(client_fd, buffer, sizeof(buffer))) > 0) {
        ssize_t bytes_written = 0;
        // Подсигуряваме пълното изпращане на прочетения блок данни
        while (bytes_written < bytes_read) {
            ssize_t res = write(client_fd, buffer + bytes_written, bytes_read - bytes_written);
            if (res <= 0) {
                break; // Грешка при писане или затворен сокет
            }
            bytes_written += res;
        }
    }

    close(client_fd);
    exit(0); // Детето приключва работа успешно
} else {
    // ----- БАЩИН ПРОЦЕС -----
    close(client_fd); // Бащата затваря клиентския сокет и веднага се връща към accept()
}

close(server_fd);
return 0;
}

```

Прочитане на съобщение от клиент и изпращане обратно към клиент.